# OsdagBridge Report Generator — Technical Changes

Nov 13, 2026 · @Developers

All six required report-generator tasks are complete in seven commits (one per task, plus a Task 6 follow-up) on branch `report-enhancements`, based on commit `d74f56d` of the `Delishaduarte/OsdagBridge` fork. Every value in the report and its graphs still comes from the design inputs and outputs; no engineering value is hard-coded.

## At a glance

The sample bridge's report grows from 56 to 58 pages, and its LaTeX errors fall from 140 to 0.

| Task | Commit | What changed | Main files |
|---|---|---|---|
| 1. Repeated table headers | `66d9ab1` | 49 tables repeat their headings on every continuation page, with "continued" markers | chap2–chap7, report_generator |
| 2. Footer overlap and layout | `43d85ab` | No text under the footer or past the margins; groups and headings kept together; page numbering fixed | report_generator, chap2/4/5/8, executive_summary, report_utils |
| 3. Vehicle and footway loads | `ffe38ef` | Two separate tables; braking force calculated; centrifugal force reported honestly | chap3, irc6_2017 |
| 4. Utilization ratio graphs | `769c1bf` | Figures 5.1 and 5.2 in Section 5.5; end diaphragm shows its own data | chap5, report_charts (new), report_generator |
| 5. Material quantity graphs | `30f74d6` | Figures 7.1 and 7.2 in Chapter 7; end-diaphragm steel added to the bill of materials | chap7, report_charts, boq_generator |

| Task | Commit | What changed | Main files |
|---|---|---|---|
| 6. Centralised styling | `b0d068d` | Every report-wide style value moved into `styles.py` | styles (new) and all report files |

New unit tests: `tests/unit/test_irc6_braking.py` (7) and `tests/unit/test_boq_end_diaphragm.py` (4); all 11 pass.

## Task 1 — Repeated table headers

Column headings now repeat on every page a table continues onto. The original code already used `longtable` for most tables, but only 2 of 50 had a working `\endhead`.

- **Header blocks:** 49 longtables in Chapters 2, 3, 4, 5 and 7 got `\endfirsthead` / `\endhead` blocks. The conversion was scripted, so every table follows the same pattern.
- **Continuation markers:** each continued page starts with "Table X.Y (continued)", and each break ends with "Continued on next page". These are two new preamble macros, `\ltcontinuedhead` and `\ltcontinuedfoot`.
- **Caption syntax:** most captions were missing the `\\` that `longtable` requires after `\caption`. Adding it cleared 141 LaTeX errors in the original report.
- **Table 2.8:** its `\endhead` was empty, so nothing repeated; fixed.
- **Table 7.1:** a bold text line stood in for a caption; it is now a numbered caption.
- **Chapter 7 scoping bug:** an unclosed `\begingroup` leaked narrow cell padding into Chapters 8–9; it is now closed.
- **Captions stranded below the next page's header:** `longtable` judges whether a table fits by measuring only a sliver of its first row. Tall first rows therefore started with no room, leaving an empty footer behind. A small patch to `\LT@start` reserves extra lines in that check (`\LTstartreserve`).

Two short tables (Table 1.1 and the small Chapter 2 info tables) always fit on one page, so they stay as plain `tabular`.

## Task 2 — Footer overlap and page layout

No body text now reaches the footer or crosses the right margin, measured on every page of four bridge configurations.

| Measure | Original | After Task 2 |
|---|---|---|
| Table rows wider than the text area | 210 (up to 59 pt / 21 mm over) | 0 (largest 0.3 pt) |
| Minimum clear space above the footer | 15 pt | 28 pt |
| "Page X of Y" correct on every page | No (off by one) | Yes |
| LaTeX errors (sample bridge) | 140 | 0 |

Causes found and fixed:

- **Footer note in the text area:** the "* Software default value" note was pushed up into the text block with negative `\vspace`. It now sits inside the footer zone.
- **Header too small:** `fancyhdr` warned that `headheight` was 12 pt where 17.5 pt was needed; it is now 18 pt.
- **Margins:** A4 with 25 mm top/bottom and 20 mm sides (was 1 inch all round), which gives wide tables 15 mm more room.
- **Leaking table settings:** a Chapter 2 table set `\tabcolsep` and `\LTleft` without a group, so every later table inherited them. All per-table settings are now scoped to their own table.
- **Over-wide tables:** column specs of the bracing and girder tables are narrowed. The Executive Summary now sizes its columns from `\linewidth` instead of an assumed 15 cm. Chapter 8 uses `tabularx`, and two of its unclosed groups are now closed.
- **Long words in narrow cells:** `L`, `C` and `R` columns now allow hyphenation of a cell's first word, and `\emergencystretch` lets lines loosen instead of overflowing.
- **Split row groups:** a new `keep_rows_together()` helper and a non-breaking `\nbcline` keep each girder's or bracing pair's rows on one page, so labels like "G4" are no longer stranded.
- **Stranded headings:** `\needspace` before `\section` and `\subsection` keeps a heading with the content below it. The old `\needspace` hook on longtables broke `longtable`'s own fit check and was removed.
- **Page numbering:** `titlepage` reset the counter to 1, so numbering now restarts at 2 after the title page.
- **Missing symbols:** ⩽, ⩾, ≠, ≈ and subscript s were silently dropped by pdflatex; they are now declared and print.

Trade-off: keeping girder groups whole adds two pages to the sample report.

# Task 3 — Vehicle and footway live loads

Chapter 3's single "Live Loads" table is now two: Table 3.3 Vehicle Live Loads and Table 3.4 Footway Live Loads. Both are filled from the UI inputs and the IRC 6 functions.

| Table | Row | Source |
|---|---|---|
| Vehicle | Vehicles considered | UI vehicle selections, including custom vehicles |
| Vehicle | Number of design lanes | UI lane table if filled; otherwise IRC 6 Table 6 from carriageway width (the rule the analyser uses) |
| Vehicle | Impact factor | IRC 6 Cl. 208.2 / 208.3, per selected vehicle class |
| Vehicle | Braking load | IRC 6 Cl. 211.2, in kN and t |
| Vehicle | Centrifugal force | "Not applicable – straight bridge" (see below) |
| Footway | Footpath provided, loading mode, intensity, unit | `fp_str` (IRC 6 Cl. 206.1, 4.905 kN/m², or the user value) |

**Braking force.** `cl_211_2_braking_force` in `irc6_2017.py` crashed for every lane count, so the report always showed "N/A". Following the agreed steps:

1. The excess-lane term `braking_force_2` was never initialised before the loop.
2. For 1–2 lanes, Cl. 211.2 makes that term genuinely zero, so it now starts at 0.0. A lane count below 1 now raises `ValueError` instead of returning a value.
3. The formula itself is unchanged.
4. Results match hand calculations: 108.69 kN for 1–2 lanes and 157.74 kN for 3 lanes. These are now unit tests.
5. Missing inputs show as "Not available (reason)", never 0.

The same check found a units error. The function returns newtons (vehicle loads are stored in N), but Chapter 3 treated the result as tonnes and multiplied by 9.81. It now converts N to kN and t correctly.

**Centrifugal force.** The backend does not calculate it and has no curve radius or design speed input, so the table says it is not applicable rather than inventing a number.

**Footway value.** The analysis always applies the IRC 6 Cl. 206.1 value. When a user enters a custom footway load, the report now says the analysis used 4.905 kN/m².

Tested with all vehicles, Class A only, no footpath, a user-defined footway load and a 3-lane deck.

## Task 4 — Utilization ratio graphs

Section 5.5 now has two Matplotlib figures, drawn from exactly the URs printed in Table 5.26 (Overall Design Check Summary).

- **Figure 5.1 — overall UR per element:** Steel Plate Girders, Concrete Deck Slab, Cross Bracing and End Diaphragms. It has a red dashed line at UR = 1.0, a PASS/FAIL label and value on each bar, and the governing check named under each element. FAIL bars are hatched so they read in greyscale print.
- **Figure 5.2 — every check:** a horizontal bar per check, grouped by element.

**Overall UR of an element = the highest UR among its checks.** An element is only as safe as its governing check, so a failure in any one check marks the whole element as failing. The report states this under the 5.5 heading.

How the data flows:

- **Recording URs:** the summary-table row builders in `chap5.py` now also return each check's numeric UR, recorded as `ur_records` (element, check, UR). The graph is drawn from that list, so it cannot disagree with the table.
- **Missing values:** the table's builders turned a missing UR into 0.0. The graph receives the raw value instead, so a missing UR is never plotted as a passing 0.
- **Failure handling:** chart code lives in the new `report_charts.py`. Any plotting failure is logged and the figure is left out; the report still compiles.

**End diaphragm.** Section 5.4 and the summary rows were showing the intermediate cross-bracing numbers as end-diaphragm results; the capacity table literally reused `cb_capacity_content`.

- **Real data:** a new `ReportDataBridge.end_diaphragm_view()` reads the backend's own `end_diaphragm_forces_dict` and `end_diaphragm_design_results` through the same accessors. The three bracing table builders are now one function, called once per bracing system.
- **No-force case:** where the analysis gives the end diaphragm no force (see Backend findings), Section 5.4 explains why, and the graph draws an empty "No force from analysis" outline instead of 0.
- **Labels:** end-diaphragm rows for bracing-type diaphragms are now labelled Compression / Tension / Slenderness, not Moment / Shear.

Also fixed: bracing connection tables printed each panel label twice, and the effective length showed 0 when the geometry was not stored (now "---").

**Verification.** On two bridges, all 33 plotted URs were compared automatically with Table 5.26 and with Demand ÷ Capacity from the same row, and each element bar with the maximum of its checks: 0 mismatches. On the 15° skew bridge the end diaphragm does carry force, and its bar (0.98) comes from end-diaphragm data, not cross bracing (0.96).

## Task 5 — Material quantity graphs

Chapter 7 now has two figures below Table 7.1, built on the existing `calculate_material_quantities()` pipeline rather than a new calculator.

- **Figure 7.1 — structural steel by element:** girders, cross bracing (top + bottom + diagonals) and end diaphragms, with MT and share of total. The caption gives the total, 43.08 MT on the sample bridge.
- **Figure 7.2 — concrete and reinforcement:** concrete volume (deck slab, crash barriers, $m^3$) and reinforcement weight (deck slab, MT). The units differ, so they are two side-by-side panels with their own axes, never one chart with two y-axes. The deck's reinforcement ratio is shown as about 120 $kg/m^3$.

Data rules:

- **Same values as the table:** the quantities arrive as formatted strings. The graphs parse the exact strings Table 7.1 prints (`parse_quantity`).
- **Missing values:** "N.A." stays missing and is drawn as "Not calculated", never as 0. An element with any missing part is not partially summed.
- **No data at all:** if nothing is available, the figure is omitted and the chapter still compiles.
- **Table code:** Table 7.1's eleven rows are now generated from one list. The original eight rows are byte-for-byte unchanged, and the table has a label for cross-references.

**End-diaphragm steel (`boq_generator.py`).** The calculator had no end-diaphragm quantity, but the brief's steel graph needs one. New rows 2(d)–2(f) cover the end-diaphragm top chord, bottom chord and diagonals:

- **Method:** the same as the intermediate bracing (area × length × 7.85 $t/m^3$), at both supports of every girder pair.
- **Geometry:** the backend's own end-diaphragm geometry: depth h = 0.85 × girder depth, width = girder spacing, diagonal = $\sqrt{(s^2 + h^2)}$ for X-bracing, using s/2 for K-bracing.
- **Section data:** areas come from the design output (`transverse_member_design.ed.section_properties.*`).
- **Edge cases:** disabled chords count as 0. Missing data or Rolled/Welded end diaphragms give "N.A.", since there is no backend design to test those against.

- **Error handling:** the helper catches its own errors, so a problem cannot skip the crash-barrier step after it.

Verified against a hand calculation on the sample bridge: chords 8 × 0.29 MT each side, diagonals 16 at 2.8142 m = 0.68 MT. Four unit tests cover this plus the disabled-chord, missing-area and rolled-beam cases. On four bridges, each caption total equals the Table 7.1 sum.

## Task 6 — Centralised styling (`styles.py`)

`src/osdagbridge/core/reports/styles.py` is now the single source of truth for how the report looks. Changing a value there changes it everywhere, charts included.

| Group | Settings in `styles.py` |
|---|---|
| Page | Paper size, four margins, header height and separation, footer skip |
| Fonts | Base size (12 pt), family (Computer Modern or Times), line spacing (1.15), heading sizes, chapter heading spacing |
| Colours | Osdag green rules, heading green, table-header fill, text, PASS and FAIL |
| Tables | Cell padding (normal, compact, dense), row height, rule width, row gaps (normal, small, large), longtable spacing and start reserve, keep-together lines |
| Spacing | Block gap, small gap |
| Captions | Label bold, table vs figure caption weight, justification |
| Header / footer | Footer text, rule widths, software-default note |
| Charts | DPI, PASS / FAIL / limit / quantity colours, ink, grid, font size; `chart_rc()` matches the report font |

**How it works.** The constants are plain Python. `latex_style_definitions()` turns them into named LaTeX colours, lengths and macros, so chapters use names instead of numbers: `\osdagRowGap`, `\osdagBlockGap`, `\osdagColSepCompact`, `\osdagHeadRow`, `\osdagPassText`, `\osdagFailText`, `\osdagUnnumberedHeading`. `report_charts.py` reads the same constants.

**Migrated, not just added.** The old values were removed, not left beside the new file:

- 266 table row gaps ( `\\[6pt]` and similar)
- 7 local cell-padding overrides; the four 4 pt ones were redundant and removed
- 55 vertical gaps
- 16 PASS/FAIL colours
- 55 table captions with hand-written bold, now set once by `\captionsetup[table]`
- The preamble's colours, geometry, table spacing, captions, keep-together rules, header/footer rules and line spacing
- The chart colours, DPI and font

The migration script handled raw and normal Python strings separately, since LaTeX backslashes are escaped differently in each. Every report file then compiled with Python's escape warnings treated as errors. About 20 one-off positioning nudges remain local (title-page placement, figure placement, the Chapter 2 diagram layout), because they place specific elements rather than set report-wide style.

**Bugs fixed on the way.** The chapter-heading style lived inside `toc_section()`, so turning off the table of contents changed every chapter heading. The Executive Summary also redefined it with different spacing for all later chapters. It is now defined once.

**Times font option.** Setting `FONT_FAMILY = "times"` switches body text and charts to Times. It compiles cleanly but re-flows pages and strands one heading on the sample bridge, so Computer Modern stays the tested default.

**Follow-up check (commit `2d1f326`).** To confirm Task 6 works, eight values were changed in `styles.py` only and the report was regenerated: heading colour, table-header fill, FAIL colour, left margin, row gap, footer text, chart bar colour and chart font size. In the generated LaTeX the differences were confined to the preamble's style block. Every change appeared in the PDF, but the chart font size had almost no effect, because `report_charts.py` still hard-coded 15 font sizes. Fixed:

- **Chart text sizes:** all now derive from `CHART_FONT_SIZE`, through four offsets in `styles.py` ( `CHART_FONT_SIZE_SMALL`, `_NOTE`, `_LABEL`, `_TITLE` ).
- **Figure 5.1 labels:** wrapped by measuring the rendered text, so neighbouring labels never collide at a larger font. Words are never split; if one word per line still does not fit, the figure widens.
- **No-force note:** "No force from analysis" has a white backing, so it stays readable if it reaches the UR = 1.0 line.

Checked at chart font sizes 9, 12 and 14. At the default size (9) the report is unchanged: 58 pages, same figures.

# Testing

The final code passes every check below on all four bridge configurations.

| Configuration | Inputs changed from the sample | Pages |
|---|---|---|
| Sample bridge | 30 m span, 7.5 m carriageway, footpaths both sides, 0° skew, Mumbai | 58 |
| Skewed | 40 m span, 15° skew | 57 |
| Wide deck | 11 m carriageway (3 lanes, 7 girders) | 64 |
| Short, no footpath | 20 m span, no footpath | 53 |

Checks run on every generated PDF:

- pdflatex errors, read from LaTeX's own log: 0
- Text past the right margin: 0 pages
- Body text below the text area, measured by word position: none closer than 28 pt to the footer
- Headings stranded at a page bottom: 0
- Captions orphaned from their continued table: 0
- "Page X of Y" matching the PDF page on every page: yes
- Every "Continued on next page" followed by a "(continued)" header, and vice versa: yes

Task-specific checks:

- **Task 3:** vehicle selection checked with all vehicles and with Class A only.
- **Task 4:** URs plotted vs table vs Demand ÷ Capacity, 33 values, 0 mismatches.
- **Task 5:** caption totals vs Table 7.1 sums, all four bridges.
- **Task 6:** the eight original Table 7.1 rows byte-identical after the rewrite.

Unit tests: 11 pass ( `tests/unit/test_irc6_braking.py` , `tests/unit/test_boq_end_diaphragm.py` ).

`Report_Before.pdf` is commit `d74f56d` run on the same sample bridge in the same environment, so the comparison with `Report_After.pdf` is like for like.

Two existing 2 mm overruns of the bold word "Longitudinal" in a narrow Chapter 2 column remain. They stay inside the cell padding and were left alone.

# Backend findings for the team

These were found while tracing report data and deliberately not changed, because they alter engineering results or need a design decision.

| Finding | Where | Effect today | Suggested owner action |
|---|---|---|---|
| Grillage restrains every girder end laterally (pin `[1,1,1,0,0,0]` and roller `[0,1,1,0,0,0]` both fix Z) | `ospgrillage` edge supports, used by `analyser.py` | End diaphragms on straight bridges get zero horizontal force, so they are never designed. The bearing drawing (`builder.py`) says only guided girders are restrained sideways. | Decide the intended bearing restraints and model them; this changes all analysis results |
| Braking force adds the 5% excess-lane term once, not per extra lane | `irc6_2017.py`, `cl_211_2_braking_force` | 4 and 6 lanes give the same value as 3 (157.74 kN). IRC 6 Cl. 211.2(b) applies it to each lane beyond two. | Confirm the intended formula; the crash fix kept it as written |
| User-defined footway load is ignored by the analysis | `analyser.py`, `create_footpath_load` | The analysis always applies 4.905 kN/m² (IRC 6 Cl. 206.1). The report now states this when the values differ. | Pass the user value into the load case if custom loads are intended |
| Wind and seismic values blank in Tables 3.5 and 3.6 (Vz, Pz, forces, Z, Sa/g, Ah, Av) | Values not reaching `input_dict` for Chapter 3 | Several rows print empty | Trace where these computed values are stored |
| Centrifugal force not modelled | No curve radius or design speed input | Reported as not applicable | Add only if curved bridges are in scope |

| Finding | Where | Effect today | Suggested owner action |
|---|---|---|---|
| Bracing top-chord quantity uses the diagonal's section area | `boq_generator.py`, cross-bracing rows | Correct only while chord and diagonal sections are the same | Use the chord section's own area |

Environment note: the main Osdag package's steel-section database (`Intg_osdag.sqlite`) ships empty from source and must be built from `Intg_osdag.sql`. Without it every cross-bracing design returns nothing. A conda install of Osdag already includes it.

## Reproducing the reports

The reports were generated headlessly by a small runner, `tools/run_report.py`. It is not committed, because the repository's `.gitignore` excludes `tools/`. It mirrors the GUI flow:

1. Start from `BASIC_INPUT_DICT`.
2. Fill the required fields the GUI asks for: project location, span, carriageway width, footpath, skew.
3. Fill material constants from the material database, the same way the Material Properties dialog does (G = E / 2(1 + ν)).
4. Run the layout solve, then `PlateGirderBridge.design()`.
5. Call `generate_design_report()` with every chapter selected.

Overrides on the command line test other configurations, for example `geometry.span=40` `geometry.skew_angle=15`. `True` / `False` become real booleans, as the GUI checkboxes produce.

Environment used: Python 3.12, `pip install -e .` plus `pylatex`, TeX Live 2023 (`longtable` v4.19), and the Osdag `dev` branch installed for `osdag_core` with its database built. Matplotlib 3.11 renders the charts with the `Agg` backend, which needs no display.